

Why Full Fine-Tuning is Impractical

Full fine-tuning means updating **all weights** of the model during training. For a Llama 3 8B model, that's 8 billion parameters $\times$ 4 bytes (FP32) = **32GB VRAM** just to store the model.

But that's just the start. Add optimizer states, gradients, and activations during training, and you need **80-120GB VRAM**. That's 3-4 high-end A100 GPUs costing over **\$10,000+**.

For most researchers and practitioners: completely inaccessible

80-120GB

VRAM Required

Full fine-tuning of Llama 3 8B

\$10K+

Hardware Cost

3-4 A100 GPUs

3-4x

VRAM Reduction

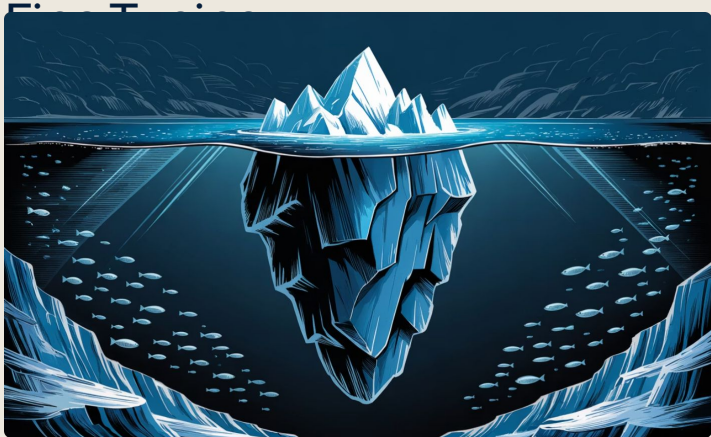
With PEFT methods



The question: Can we get 90% of fine-tuning benefit at 5% of the cost?

The answer: Yes — Parameter Efficient Fine-Tuning (PEFT)

PEFT: Parameter-Efficient



Frozen Base Model

Retains all general knowledge from pre-training

Small Adapter

Teaches specific task without modifying base

10–100× Fewer

Trainable parameters means fraction of memory and compute cost

PEFT methods include: LoRA (most popular), Prefix Tuning, Prompt Tuning, Adapter Layers

HuggingFace PEFT library: `pip install peft` — unified interface for all PEFT methods

Don't rewrite the encyclopedia. Add a specialist appendix.

The Core Idea

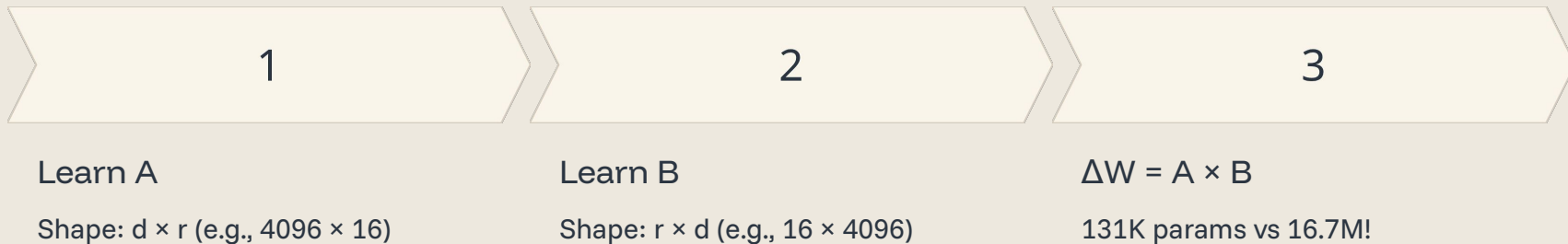
Freeze the original model weights. Only train a small set of additional parameters.

Original model weights stay **frozen** (not updated, not stored in gradient memory). Only the new adapter parameters get updated during training.

LoRA: Low-Rank Adaptation

The Intuition

Full fine-tuning updates weight matrix W (e.g., $4096 \times 4096 = 16.7\text{M}$ parameters). **LoRA insight:** Weight updates during fine-tuning have **low intrinsic rank** — they don't need full matrix updates.



During inference: $W_{\text{new}} = W_{\text{frozen}} + \Delta W$ — merged, no extra latency.

Where LoRA is applied: Query (q_{proj}), Key (k_{proj}), Value (v_{proj}) weight matrices in self-attention layers

Key hyperparameters: r (rank — higher = more expressive but more memory), α (scaling factor $\approx 2 \times r$)

LoRA in Code

HuggingFace PEFT Implementation

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=16,                # rank
    lora_alpha=32,        # scaling factor
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
```

Output

```
trainable params: 4,194,304
all params: 6,742,609,920
trainable%: 0.062%
```

Saving

```
model.save_pretrained()
```

Saves only ~50MB, not 13GB

Merging

```
model.merge_and_unload()
```

Combine adapter with base model

Quantization: Shrinking the Model

The Problem

Even loading a 7B model requires ~14GB VRAM (FP16). Most researchers have 8–16GB GPUs.

The Solution

Reduce the numerical precision of weights to use less memory.



01

FP32 (Full Precision)

4 bytes per weight — default training format

03

INT8

1 byte per weight — 4× smaller than FP32, small accuracy loss

02

FP16 / BF16 (Half Precision)

2 bytes per weight — standard inference

04

INT4 / NF4

0.5 bytes per weight — 8× smaller, moderate accuracy loss



Trade-off: Precision ↓ → Memory ↓ → Slight accuracy ↓

Quantization types: PTQ (Post-Training Quantization) — train in FP32, convert after. QAT (Quantization-Aware Training) — simulate quantization during training for better accuracy.

7B model in FP32 = 28GB

Same model in INT4 = ~3.5GB

Fits on laptop GPU

QLoRA: Fine-Tuning on a Laptop GPU

QLoRA = Quantization + LoRA combined (Dettmers et al., 2023)



Load base model in 4-bit precision (NF4)

Saves 8× memory vs FP32



Use double quantization

Saves another 0.37 bits/param



Apply LoRA adapters

Only train small adapter matrices



Compute in BF16

For numerical stability despite 4-bit storage



Result: Fine-tune a 7B LLM on a single 16GB GPU (Google Colab free tier!)

BitsAndBytes Config

```
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config
)
```

Full Pipeline: QLoRA End-to-End

Complete Workflow in 6 Steps

01

Load tokenizer and dataset

Using HuggingFace datasets library

03

Apply LoRA config

Use `get_peft_model()` to wrap model

05

Train with SFTTrainer

From trl library — handles instruction tuning format

02

Load base model with BitsAndBytesConfig

4-bit quantization applied at load time

04

Define TrainingArguments

Set lr, epochs, batch size, output directory

06

Save and optionally merge

Save adapter → merge with base model

Tools Required

- transformers
- peft
- bitsandbytes
- trl
- datasets

All free, all HuggingFace ecosystem



Hardware requirements: Google Colab free tier (T4 GPU, 15GB VRAM) is sufficient for 7B models with QLoRA

Choosing Your Strategy

Decision Framework

Approach	VRAM Needed	Time	Cost	When to Use
Zero-shot / Few-shot Prompting	0 (API)	Minutes	Free	Quick tasks, general domains
Full Fine-Tuning	80–120GB	Days	\$500+	Maximum performance, large budget
LoRA Fine-Tuning	20–40GB	Hours	\$20–100	Good performance, moderate resources
QLoRA Fine-Tuning	8–16GB	Hours	\$5–20	Best bang for buck, Colab compatible
RAG (Retrieval)	Low	Minutes to setup	~Free	Domain knowledge, no retraining

☐

Do you have labeled domain data?

No → Use RAG or prompting

☐

Do you need consistent instruction following?

Yes → LoRA/QLoRA

☐

Do you need to inject private knowledge?

Frequently changing? → RAG wins